

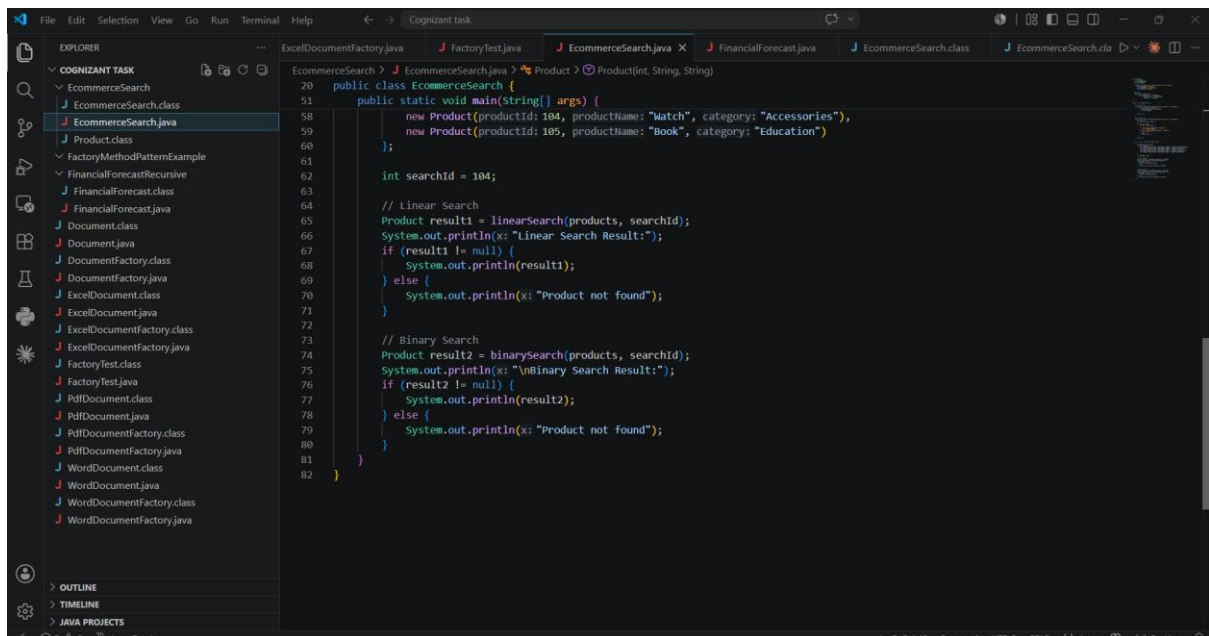
Data structures and Algorithms

Exercise 2: E-commerce Platform Search Function

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. Understand Asymptotic Notation:
 - o Explain Big O notation and how it helps in analyzing algorithms.
 - o Describe the best, average, and worst-case scenarios for search operations.
2. Setup:
 - o Create a class Product with attributes for searching, such as productId, productName, and category.
3. Implementation:
 - o Implement linear search and binary search algorithms.
 - o Store products in an array for linear search and a sorted array for binary search.
4. Analysis:
 - o Compare the time complexity of linear and binary search algorithms.
 - o Discuss which algorithm is more suitable for your platform and why.



The screenshot shows an IDE with a project named 'COGNIZANT TASK'. The Explorer panel on the left lists various Java files, including 'EcommerceSearch.java'. The main editor displays the code for 'EcommerceSearch.java', which includes a 'main' method and two search algorithms: 'linearSearch' and 'binarySearch'. The code is as follows:

```
20 public class EcommerceSearch {
21     public static void main(String[] args) {
22         new Product(productId: 104, productName: "Watch", category: "Accessories"),
23         new Product(productId: 105, productName: "Book", category: "Education")
24     };
25
26     int searchId = 104;
27
28     // Linear Search
29     Product result1 = linearSearch(products, searchId);
30     System.out.println(x: "Linear Search Result:");
31     if (result1 != null) {
32         System.out.println(result1);
33     } else {
34         System.out.println(x: "Product not found");
35     }
36
37     // Binary Search
38     Product result2 = binarySearch(products, searchId);
39     System.out.println(x: "\nBinary Search Result:");
40     if (result2 != null) {
41         System.out.println(result2);
42     } else {
43         System.out.println(x: "Product not found");
44     }
45 }
```

```
20 public class EcommerceSearch {
33     public static Product binarySearch(Product[] products, int targetId) {
37         while (low <= high) {
38             int mid = (low + high) / 2;
39
40             if (products[mid].productId == targetId) {
41                 return products[mid];
42             } else if (products[mid].productId < targetId) {
43                 low = mid + 1;
44             } else {
45                 high = mid - 1;
46             }
47         }
48         return null;
49     }
50 }

51 Run [Debug]
52 public static void main(String[] args) {
53
54     // Sorted array of products
55     Product[] products = {
56         new Product(productId: 101, productName: "Laptop", category: "Electronics"),
57         new Product(productId: 102, productName: "Mobile", category: "Electronics"),
58         new Product(productId: 103, productName: "Shoes", category: "Fashion"),
59         new Product(productId: 104, productName: "Watch", category: "Accessories"),
60         new Product(productId: 105, productName: "Book", category: "Education")
61     };
62
63     int searchId = 104;
64
65     // Linear Search
66     Product result = linearSearch(products, searchId);
67     System.out.println("Linear Search Result:");
68     if (result != null) {
69         System.out.println(result);
70     } else {
71         System.out.println("Product not found");
72     }
73 }
```

```
1 class Product {
2     int productId;
3     String productName;
4     String category;
5
6     Product(int productId, String productName, String category) {
7         this.productId = productId;
8         this.productName = productName;
9         this.category = category;
10    }
11
12    @Override
13    public String toString() {
14        return "Product ID: " + productId +
15            ", Product Name: " + productName +
16            ", Category: " + category;
17    }
18 }
19
20 public class EcommerceSearch {
21
22     // Linear Search
23     public static Product linearSearch(Product[] products, int targetId) {
24         for (Product product : products) {
25             if (product.productId == targetId) {
26                 return product;
27             }
28         }
29         return null;
30     }
31
32     // Binary Search
33     public static Product binarySearch(Product[] products, int targetId) {
34         int low = 0;
35         int high = products.length - 1;
36
37         while (low <= high) {
```

Output

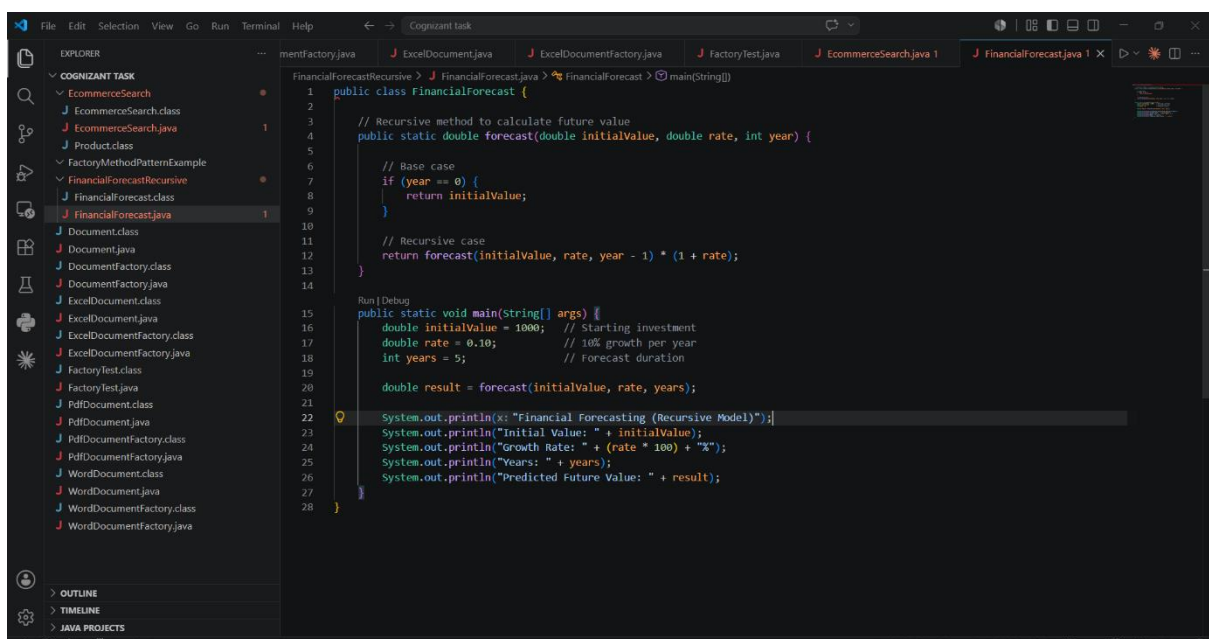
```
PS C:\Users\jyots\OneDrive\Desktop\Cognizant task> javac EcommerceSearch.java
Usage: javac <options> <source files>
use -help for a list of possible options
PS C:\Users\jyots\OneDrive\Desktop\Cognizant task> cd EcommerceSearch
PS C:\Users\jyots\OneDrive\Desktop\Cognizant task\EcommerceSearch> javac EcommerceSearch.java
PS C:\Users\jyots\OneDrive\Desktop\Cognizant task\EcommerceSearch> java EcommerceSearch
Linear Search Result:
Product ID: 104, Product Name: Watch, Category: Accessories
Binary Search Result:
Product ID: 104, Product Name: Watch, Category: Accessories
PS C:\Users\jyots\OneDrive\Desktop\Cognizant task\EcommerceSearch>
```

Exercise 7: Financial Forecasting

You are developing a financial forecasting tool that predicts future values based on past data.

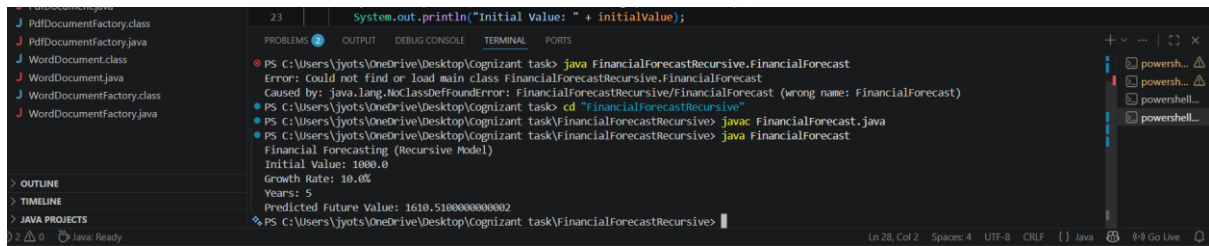
Steps:

1. Understand Recursive Algorithms:
 - o Explain the concept of recursion and how it can simplify certain problems.
2. Setup:
 - o Create a method to calculate the future value using a recursive approach.
3. Implementation:
 - o Implement a recursive algorithm to predict future values based on past growth rates.
4. Analysis:
 - o Discuss the time complexity of your recursive algorithm.
 - o Explain how to optimize the recursive solution to avoid excessive computation.



```
1 public class FinancialForecast {
2
3     // Recursive method to calculate future value
4     public static double forecast(double initialValue, double rate, int year) {
5
6         // Base case
7         if (year == 0) {
8             return initialValue;
9         }
10
11        // Recursive case
12        return forecast(initialValue, rate, year - 1) * (1 + rate);
13    }
14
15    Run | Debug
16    public static void main(String[] args) {
17        double initialValue = 1000; // Starting investment
18        double rate = 0.10; // 10% growth per year
19        int years = 5; // forecast duration
20
21        double result = forecast(initialValue, rate, years);
22
23        System.out.println(x: "Financial Forecasting (Recursive Model)");
24        System.out.println("Initial Value: " + initialValue);
25        System.out.println("Growth Rate: " + (rate * 100) + "%");
26        System.out.println("Years: " + years);
27        System.out.println("Predicted Future Value: " + result);
28    }
29 }
```

OUTPUT



```
System.out.println("Initial Value: " + initialValue);

PS C:\Users\jyots\OneDrive\Desktop\Cognizant task> java FinancialForecastRecursive.FinancialForecast
Error: Could not find or load main class FinancialForecastRecursive.FinancialForecast
Caused by: java.lang.NoClassDefFoundError: FinancialForecastRecursive.FinancialForecast (wrong name: FinancialForecast)
PS C:\Users\jyots\OneDrive\Desktop\Cognizant task> cd "FinancialForecastRecursive"
PS C:\Users\jyots\OneDrive\Desktop\Cognizant task\FinancialForecastRecursive> javac FinancialForecast.java
PS C:\Users\jyots\OneDrive\Desktop\Cognizant task\FinancialForecastRecursive> java FinancialForecast
Financial Forecasting (Recursive Model)
Initial Value: 1000.0
Growth Rate: 10.0%
Years: 5
Predicted Future Value: 1610.5100000000002
PS C:\Users\jyots\OneDrive\Desktop\Cognizant task\FinancialForecastRecursive>
```

PL/SQL programming

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

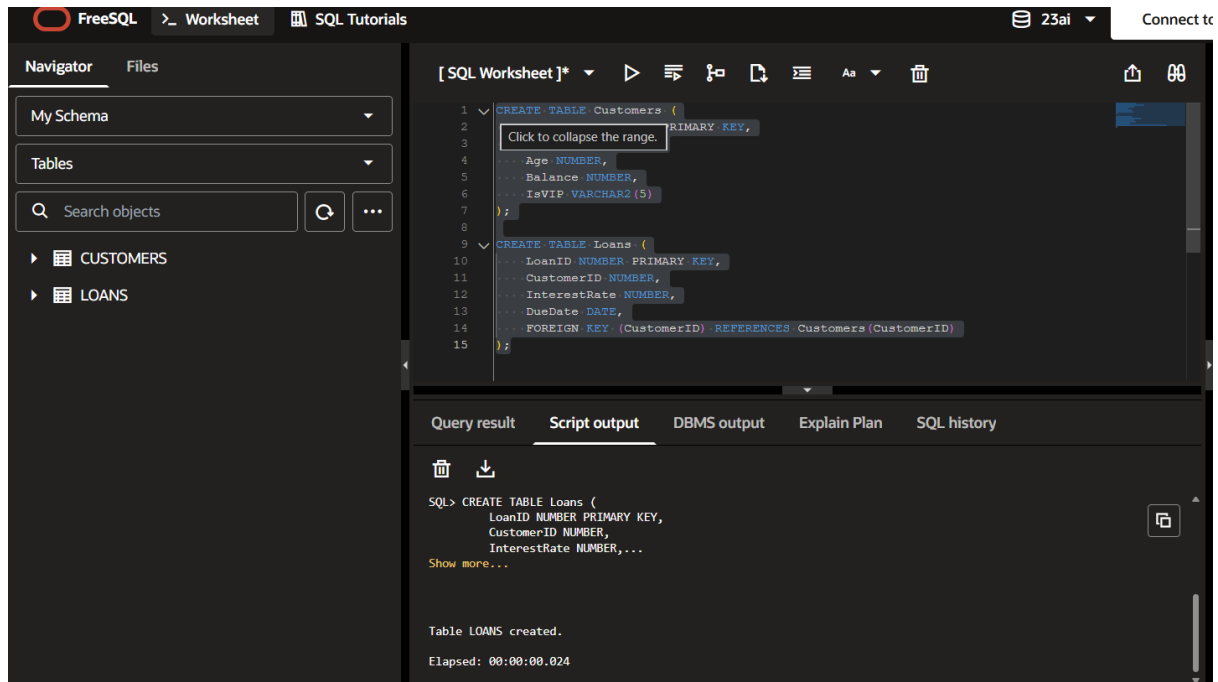
Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over \$10,000.

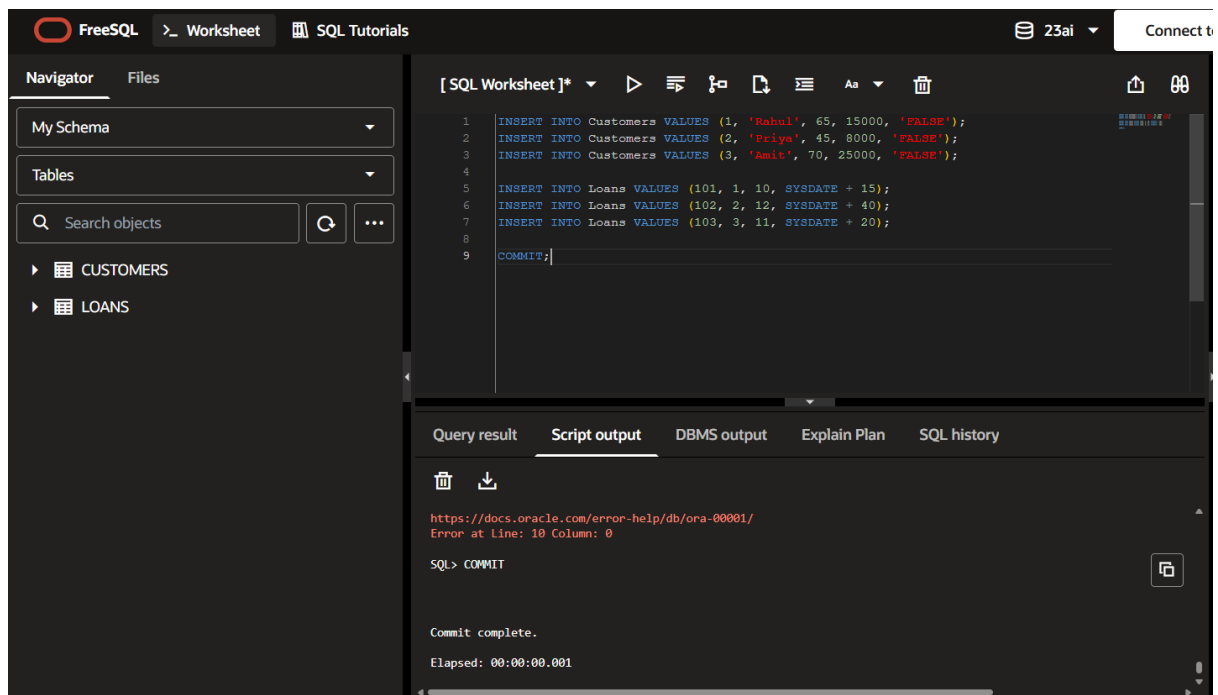
Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

Step 1: Create Tables



Step 2: Insert Sample Data



Scenario 1

Apply 1% Interest Rate Discount for Customers Above 60

FreeSQL > Worksheet SQL Tutorials 23ai Connect to

Navigator Files

My Schema

Tables

Search objects

CUSTOMERS

LOANS

```
1 BEGIN
2   FOR rec IN (
3     SELECT l.LoanID
4     FROM Loans l
5     JOIN Customers c
6     ON l.CustomerID = c.CustomerID
7     WHERE c.Age > 60
8   )
9   LOOP
10    UPDATE Loans
11    SET InterestRate = InterestRate - 1
12    WHERE LoanID = rec.LoanID;
13  END LOOP;
14
15  COMMIT;
16
17  DBMS_OUTPUT.PUT_LINE('Interest rates updated successfully.');
```

Query result Script output DBMS output Explain Plan SQL history

FROM Loans 1...

Show more...

Interest rates updated successfully.

Verify

FreeSQL > Worksheet SQL Tutorials 23ai Connect to

Navigator Files

My Schema

Tables

Search objects

CUSTOMERS

LOANS

```
1 SELECT * FROM Loans;
```

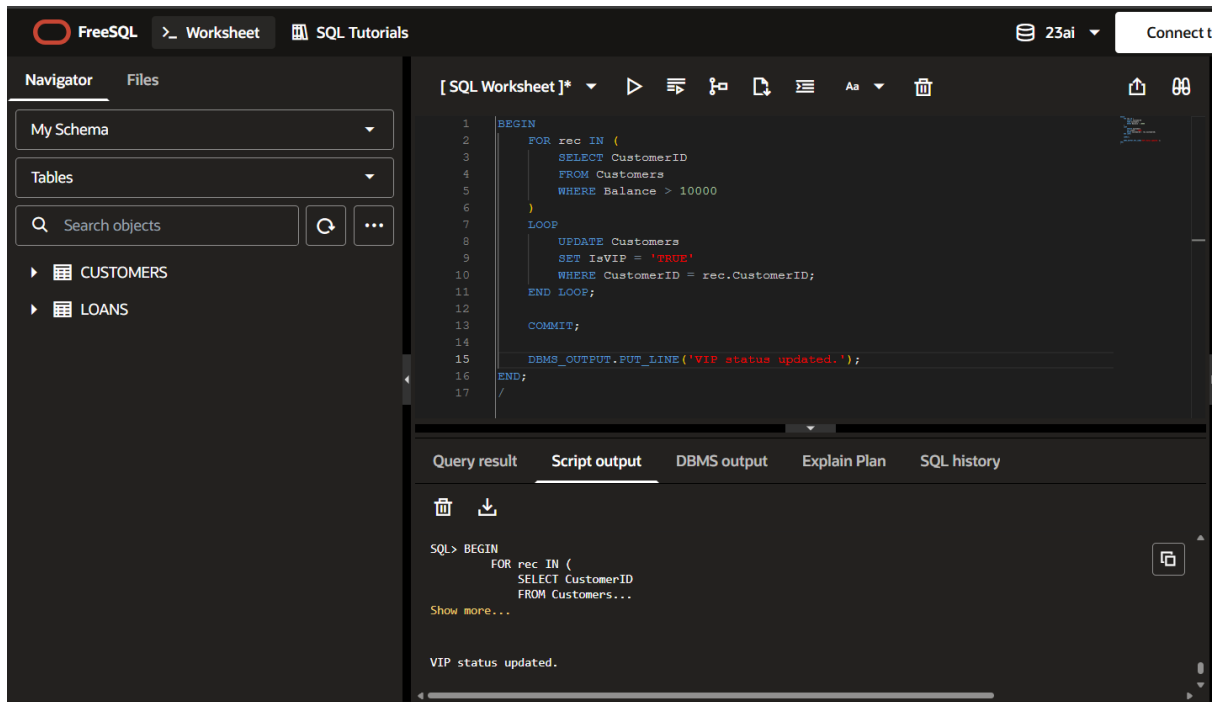
Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.006 seconds

	LOANID	CUSTOMERID	INTERESTRATE	DUE DATE
1	101	1	9	7/7/2026, 3:47:11 P
2	102	2	12	8/1/2026, 3:47:11 P
3	103	3	10	7/12/2026, 3:47:11

Scenario 2

Set VIP Status for Customers with Balance > 10000



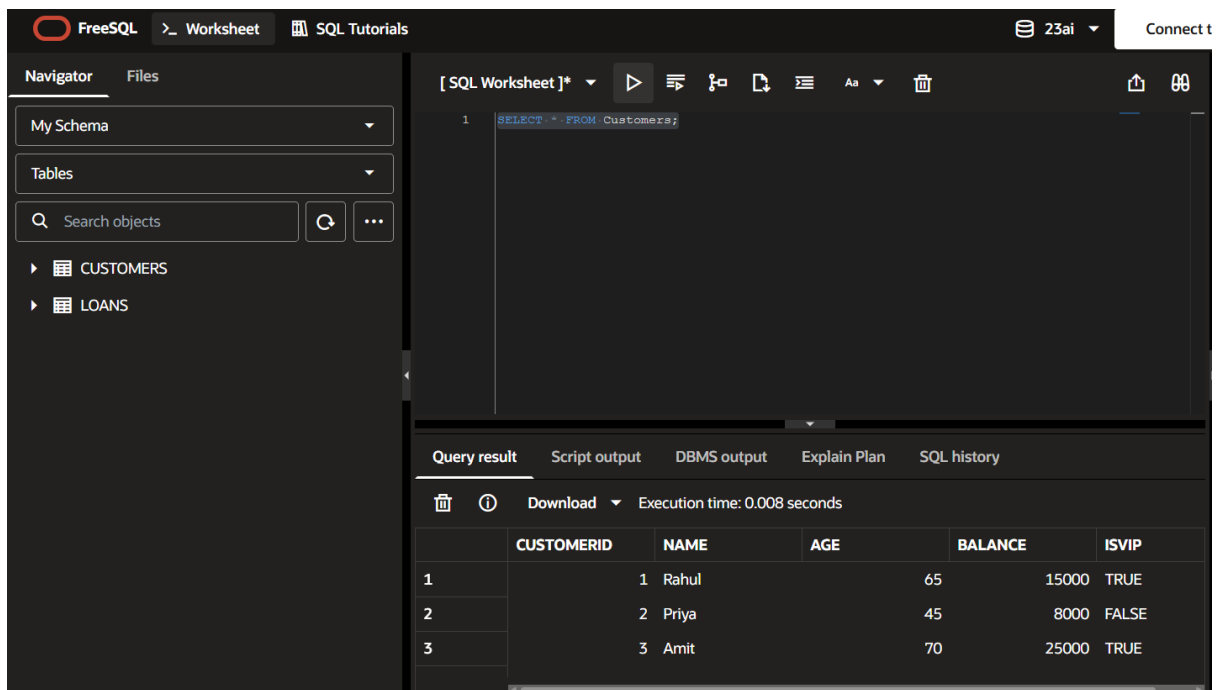
The screenshot shows the FreeSQL IDE interface. On the left, the Navigator pane displays 'My Schema' and 'Tables' with a search bar. The main editor shows a SQL script for updating customer VIP status. The script uses a loop to iterate over customers with a balance greater than 10,000 and sets their 'IsVIP' status to 'TRUE'. The script output pane at the bottom shows the execution progress and a confirmation message.

```
1 BEGIN
2   FOR rec IN (
3     SELECT CustomerID
4     FROM Customers
5     WHERE Balance > 10000
6   )
7   LOOP
8     UPDATE Customers
9     SET IsVIP = 'TRUE'
10    WHERE CustomerID = rec.CustomerID;
11  END LOOP;
12
13  COMMIT;
14
15  DBMS_OUTPUT.PUT_LINE('VIP status updated.');
```

Query result | **Script output** | DBMS output | Explain Plan | SQL history

SQL> BEGIN
FOR rec IN (
 SELECT CustomerID
 FROM Customers...
Show more...
VIP status updated.

Verify



The screenshot shows the FreeSQL IDE interface with the 'Query result' pane selected. The main editor shows a simple SQL query: 'SELECT * FROM Customers;'. The query result pane displays the execution time and a table of customer data.

```
1 SELECT * FROM Customers;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download Execution time: 0.008 seconds

	CUSTOMERID	NAME	AGE	BALANCE	ISVIP
1	1	Rahul	65	15000	TRUE
2	2	Priya	45	8000	FALSE
3	3	Amit	70	25000	TRUE

Scenario 3

Send Loan Due Reminders

The screenshot shows the FreeSQL IDE interface. On the left, the 'Navigator' pane displays 'My Schema' and 'Tables'. Under 'Tables', there are two tables: 'CUSTOMERS' and 'LOANS'. The main editor area shows a PL/SQL procedure named 'SEND_LOAN_DUE_REMINDERS'. The procedure uses a cursor to iterate over loans that are due within the next 30 days. For each loan, it prints a reminder message to the console. The output pane at the bottom shows the results of the procedure execution, displaying two reminder messages: 'Reminder: Loan 101 for Rahul is due on 07-JUL-2026' and 'Reminder: Loan 103 for Amit is due on 12-JUL-2026'. The status bar at the bottom indicates 'PL/SQL procedure successfully completed.'

```
1 BEGIN
2   FOR rec IN (
3     SELECT c.Name, l.LoanID, l.DueDate
4     FROM Customers c
5     JOIN Loans l
6     ON c.CustomerID = l.CustomerID
7     WHERE l.DueDate BETWEEN SYSDATE AND SYSDATE + 30
8   )
9   LOOP
10    DBMS_OUTPUT.PUT_LINE(
11      'Reminder: Loan '
12      || rec.LoanID
13      || ' for '
14      || rec.Name
15      || ' is due on '
16      || TO_CHAR(rec.DueDate, 'DD-MON-YYYY')
17    );
18  END LOOP;
19 END;
20 /
```

Query result **Script output** DBMS output Explain Plan SQL history

Reminder: Loan 101 for Rahul is due on 07-JUL-2026
Reminder: Loan 103 for Amit is due on 12-JUL-2026

PL/SQL procedure successfully completed.

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- Question: Write a stored procedure ProcessMonthlyInterest that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

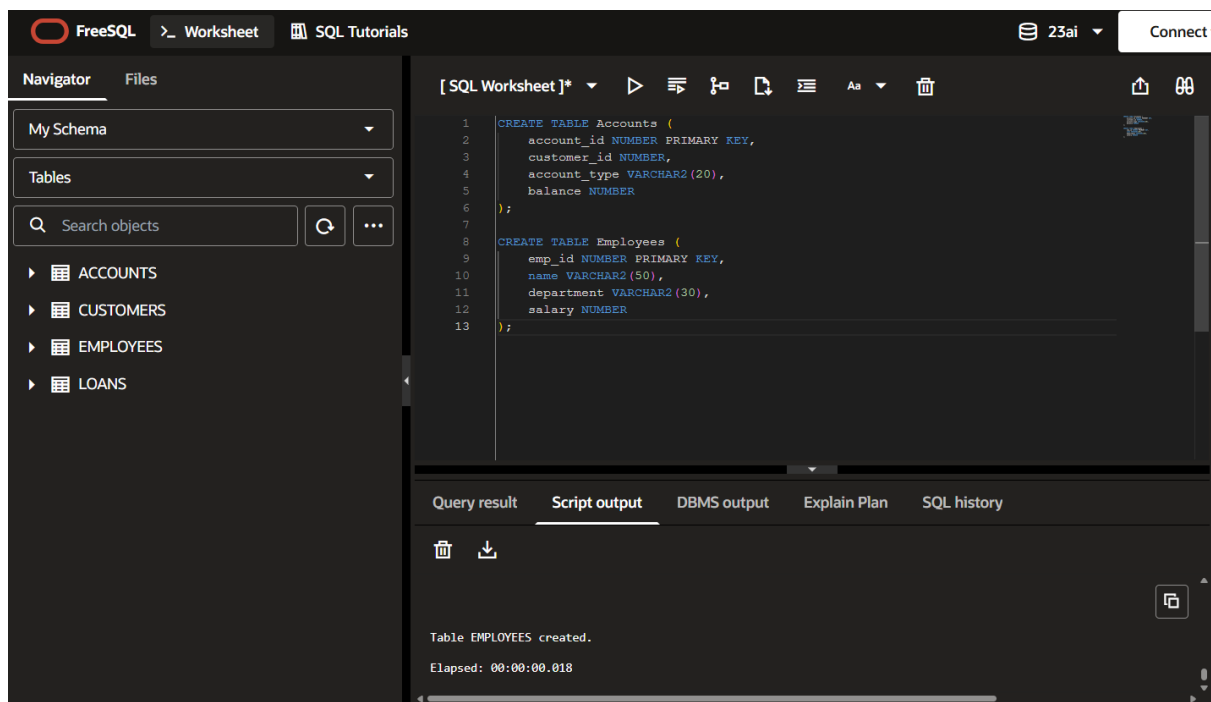
Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- Question: Write a stored procedure UpdateEmployeeBonus that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Scenario 3: Customers should be able to transfer funds between their accounts.

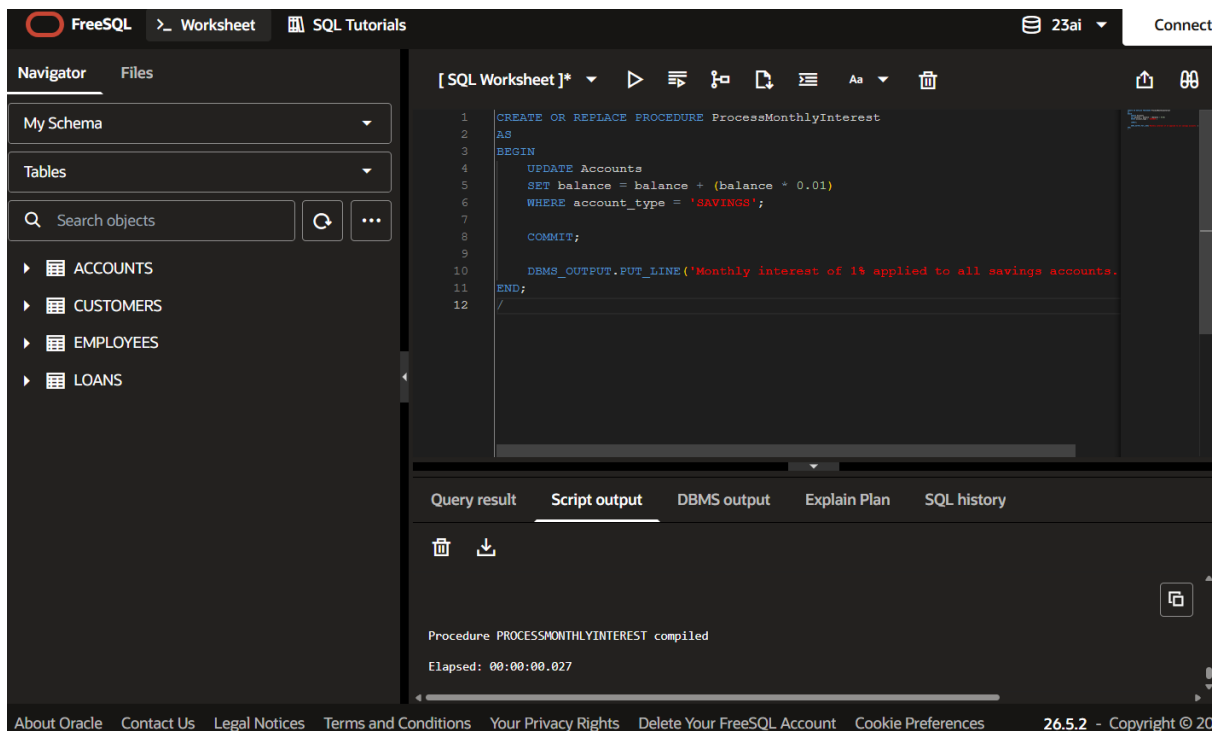
- Question: Write a stored procedure TransferFunds that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

Assumed Tables Structure

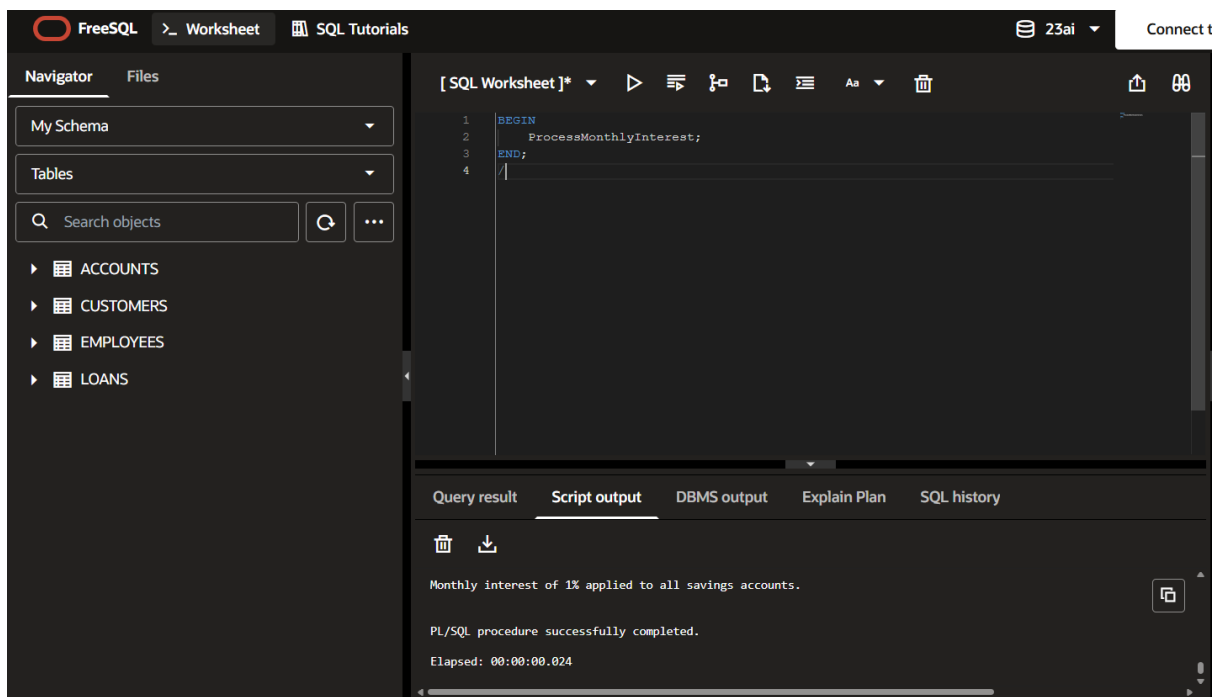


Scenario 1: ProcessMonthlyInterest

1% interest applied to all SAVINGS accounts



Execute



Scenario 2: UpdateEmployeeBonus

Adds bonus % to employees in a department

The screenshot shows the Oracle SQL Developer interface. On the left is the Navigator pane with 'My Schema' selected, showing a list of tables: ACCOUNTS, CUSTOMERS, EMPLOYEES, and LOANS. The main editor window is titled '[SQL Worksheet]*' and contains the following SQL code:

```
1 CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(  
2   p_department IN VARCHAR2,  
3   p_bonus_percent IN NUMBER  
4 )  
5 AS  
6 BEGIN  
7   UPDATE Employees  
8     SET salary = salary + (salary * p_bonus_percent / 100)  
9     WHERE department = p_department;  
10  
11   COMMIT;  
12  
13   DBMS_OUTPUT.PUT_LINE('Bonus updated for department: ' || p_department);  
14 END;  
15 /
```

Below the editor, the 'Script output' tab is active, displaying the following message:

```
Procedure UPDATEEMPLOYEEBONUS compiled  
Elapsed: 00:00:00.024
```

The bottom status bar shows '26.5.2 - Copyright © 2018'.

Execute

The screenshot shows the Oracle SQL Developer interface after executing the procedure. The main editor window now contains the following SQL code:

```
1 BEGIN  
2   UpdateEmployeeBonus('IT', 10);  
3 END;  
4 /
```

Below the editor, the 'Script output' tab is active, displaying the following messages:

```
Bonus updated for department: IT  
  
PL/SQL procedure successfully completed.  
Elapsed: 00:00:00.022
```

Scenario 3: TransferFunds

Safe transfer with balance check

The screenshot shows the FreeSQL interface with a dark theme. On the left is a 'Navigator' panel with 'My Schema' and 'Tables' sections. The main editor area is titled '[SQL Worksheet]*' and contains the following SQL code:

```
1 CREATE OR REPLACE PROCEDURE TransferFunds(  
2   p_from_account IN NUMBER,  
3   p_to_account IN NUMBER,  
4   p_amount IN NUMBER  
5 )  
6 AS  
7   v_balance NUMBER;  
8 BEGIN  
9   -- Get balance of source account  
10  SELECT balance INTO v_balance  
11  FROM Accounts  
12  WHERE account_id = p_from_account  
13  FOR UPDATE;  
14  
15  -- Check sufficient funds  
16  IF v_balance < p_amount THEN  
17    DBMS_OUTPUT.PUT_LINE('Insufficient balance. Transfer failed.');
```

This screenshot shows the same FreeSQL interface, but the SQL code is now complete. The code includes an exception block to handle cases where no data is found. The bottom panel shows the 'Script output' tab with the execution results.

```
18  ELSE  
19    -- Deduct from source  
20    UPDATE Accounts  
21    SET balance = balance - p_amount  
22    WHERE account_id = p_from_account;  
23  
24    -- Add to destination  
25    UPDATE Accounts  
26    SET balance = balance + p_amount  
27    WHERE account_id = p_to_account;  
28  
29    COMMIT;  
30  
31    DBMS_OUTPUT.PUT_LINE('Transfer successful: ' || p_amount || ' transferred');  
32  END IF;  
33  
34  EXCEPTION  
35  WHEN NO_DATA_FOUND THEN  
36    DBMS_OUTPUT.PUT_LINE('Account not found.');
```

The 'Script output' tab displays the following text:

```
Procedure TRANSFERFUNDS compiled  
Elapsed: 00:00:00.021
```

Execute

FreeSQL > Worksheet SQL Tutorials 23ai Connect to

Navigator Files

My Schema

Tables

Search objects

ACCOUNTS

CUSTOMERS

EMPLOYEES

LOANS

```
1 BEGIN
2   TransferFunds(101, 102, 5000);
3 END;
4 /
```

Query result Script output DBMS output Explain Plan SQL history

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.017

FreeSQL > Worksheet SQL Tutorials 23ai Connect to

Navigator Files

My Schema

Tables

Search objects

ACCOUNTS

CUSTOMERS

EMPLOYEES

LOANS

```
1 SELECT * FROM Accounts;
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.014 seconds

	ACCOUNT_ID	CUSTOMER_ID	ACCOUNT_TYPE	BALANCE
1	101	1	SAVINGS	10000
2	102	2	SAVINGS	20000
3	103	3	CURRENT	15000

